Instituto Tecnológico de Mexicali

Alumno y Numero de control:

José Francisco Martínez Trejo 20491190

Alejandra Medina Espinoza 21490560

Carrera:

Ing. En Sistemas Computacionales.

Materia:

Desarrollo De Aplicaciones Móviles

Grupo y Horario de clase:

B: 20:00 - 21:00

Tema:

Documentación

Docente:

Prof. José Ramon Bogarin Valenzuela

Mexicali, Baja California, 26 de septiembre del 2025

Índice

Introducción-. 4
Problemática -. 5
Justificación-. 6
Objetivo del Proyecto 7
Historial de Usuario 8
Criterios de Aceptación: 8
HU02: Buscar y Seleccionar Servicios Como cliente 8
HU03: Agendar Nueva Cita Como cliente 8
Criterios de Aceptación: 8
HU04: Ver Mis Citas Vigentes Como cliente 8
Criterios de Aceptación: 8
HU05: Cancelar o Reprogramar Cita Como cliente 8
HU06: Ver Historial de Citas Como cliente 9
HU07: Gestionar Mi Perfil Como cliente 9
USUARIO: PROVEEDOR DE SERVICIOS 9
HU09: Configurar Horario de Trabajo Como proveedor 9
HU10: Visualizar Agenda de Citas Como proveedor 10
Criterios de Aceptación: 10
HU11: Confirmar o Rechazar Citas Como proveedor 10
HU12: Gestionar Estado de Citas Como proveedor 10
Criterios de Aceptación: 10
USUARIO: ADMINISTRADOR 10
Requerimientos Funcionales Módulo de Autenticación (Sprint 1) 11
Módulo de Perfil y Gestión (Sprint 2) 11
Módulo de Servicios (Sprint 2-3) 11
Módulo de Citas (Sprint 3-4) 11
Módulo de Calendario (Sprint 4) 11
Módulo de Notificaciones (Sprint 5) 11
Requerimientos No Funcionales Usabilidad 11
Fiabilidad 12
Plan de Desarrollo por Sprints (5 Sprints) 13
Sprint 1: Base y Autenticación (Primera Entrega) 13
Sprint 2: Perfiles y Servicios (Segunda Entrega 13
Sprint 3: Núcleo de Citas (Tercera Entrega) 13

Sprint 4: Calendario e Historial (Cuarta Entrega) 13
Sprint 5: Notificaciones y Pulido (Quinta entrega) 13
Calendario 13
Pantallas. 14
Primer Sprint. 18
Presentación del tipo de arquitectura 20
Notas del Primer Sprint 25
Segundo Sprint. 25
Código del 2do Sprint 27

Introducción-.

A lo largo de este proyecto se tiene el objetivo principal el desarrollo de una aplicación móvil que permita gestionar y administrar citas para distintos tipos de servicios, esta aplicación estará diseñada para distintos usuarios, ya sea para los usuarios finales, conocidos como cliente, como también para los proveedores de los servicios, esta aplicación ofrece una plataforma intuitiva y funcional que facilite la gestión, modificación, confirmación y cancelación de citas de manera eficiente.

Esta herramienta buscara optimizar el proceso de agendado, esto con el objetivo de poder eliminar la necesidad de métodos tradicionales, que son: llamadas telefónicas o visitas al lugar para agendar, la aplicación busca que el usuario tenga mayor comodidad y autonomía, también se busca para mejorar la organización interna de los prestadores de servicios, los cuales podrán visualizar sus agendas en tiempo real, además de recibir notificaciones y poder estableces la disponibilidad, además de gestionar cambios de última hora.

Por lo tanto, este proyecto no solo se enfoca en el desarrollo técnico de la aplicación sino también busca hacer un análisis de las necesidades del usuario, la experiencia del uso, seguridad de los datos de los usuarios y escalabilidad del sistema, con el objetivo de ofrecer una solución de gestión de tiempo para diferentes servicios.

Problemática -.

En la actualidad, hay muchos negocios que ofrecen servicios por medio de citas previas, la cuales continúan manejando la reservación de forma manual o por medio de sistemas de poca eficiencia y algunos se encuentran desactualizados. Esta manera de administración puede generar una serie de problemas, tanto para los prestadores de servicio como también para los clientes estos problemas pueden ser entre, duplicación de citas, perdida de información, citas no recordadas, además que el tiempo de espera sean prolongados o que no se puedan realizar ningún cambio.

Al conocer esta problemática se busca evidencias las necesidades de una solución digital que sea accesible para el usuario y que puede optimizar la gestión de citas, además de gestionar la información y poder reducir los errores, además de buscar una mejora en la comunicación entre cliente y proveedor.

Justificación-.

En este proyecto se busca eficientica mediante una aplicación el problema de agendado de las citas donde se puedan administrar de una manera más autónoma para al clientes permitiéndoles una proyecto ambientado más a su comodidad y facilidad, tratando de incrementar la generación de citas y poder generar mejores opciones y controlar la gestión de las citas siendo un excelente servicio para el cliente gestor dándole un mayor control sobre las citas, tanto a los particulares como administrativos o empleados y a los clientes finales.

Objetivo del Proyecto

Desarrollar una aplicación móvil multiplataforma que permita a usuarios gestionar citas para diversos servicios (barbería, dentista, etc.), facilitando tanto a clientes como a proveedores la administración autónoma de citas, con el fin de optimizar el proceso de agendado, incrementar la generación de citas y mejorar la experiencia del usuario mediante una interfaz intuitiva.

Historial de Usuario

USUARIO: CLIENTE FINAL

HU01: Registro e Inicio de Sesión

Como cliente final
Quiero registrarme e iniciar sesión en la aplicación
Para acceder a todas las funcionalidades de gestión de citas
Criterios de Aceptación:

- Debo poder registrarme con nombre, correo, teléfono y contraseña
- Debo recibir confirmación de registro exitoso
- Debo poder iniciar sesión con correo y contraseña
- Debo poder recuperar mi contraseña si la olvido

HU02: Buscar y Seleccionar Servicios

Como cliente
Quiero buscar y visualizar servicios disponibles
Para seleccionar el que necesito agendar
Criterios de Aceptación:

- Debo ver un listado de servicios disponibles
- Debo poder filtrar servicios por categoría (barbería, dentista, etc.)
- Debo ver detalles del servicio (nombre, descripción, precio)
- Debo poder seleccionar un servicio para agendar cita

HU03: Agendar Nueva Cita

Como cliente
Quiero agendar una cita para un servicio específico
Para reservar mi lugar con el proveedor
Criterios de Aceptación:

- Debo seleccionar fecha y hora disponible
- Debo confirmar el servicio seleccionado
- Debo recibir confirmación inmediata de la cita
- La cita debe aparecer en mi panel principal

HU04: Ver Mis Citas Vigentes

Como cliente
Quiero visualizar mis citas agendadas
Para tener control de mis próximos compromisos
Criterios de Aceptación:

- Debo ver un listado de mis citas pendientes
- Cada cita debe mostrar: servicio, fecha, hora, proveedor
- Debo poder filtrar por fecha o estado
- Debo ver un contador de citas pendientes

HU05: Cancelar o Reprogramar Cita

Como cliente
Quiero cancelar o reprogramar una cita existente
Para ajustarme a cambios en mi disponibilidad
Criterios de Aceptación:

- Debo poder cancelar una cita con al menos 2 horas de anticipación
- Debo recibir confirmación de cancelación
- Debo poder reprogramar seleccionando nueva fecha/hora disponible
- El sistema debe notificar al proveedor de los cambios

HU06: Ver Historial de Citas
Como cliente
Quiero consultar mi historial de citas anteriores
Para llevar un registro de los servicios utilizados
Criterios de Aceptación:

- Debo ver listado de citas pasadas ordenadas por fecha
- Cada cita debe mostrar estado (completada, cancelada)
- Debo poder filtrar por servicio o fecha
- Debo poder agregar comentarios/valoraciones a citas completadas

HU07: Gestionar Mi Perfil
Como cliente
Quiero editar mi información personal
Para mantener mis datos actualizados
Criterios de Aceptación:

- Debo poder modificar nombre, teléfono y foto
- Los cambios deben guardarse inmediatamente
- Debo poder ver mi perfil público
- La información debe sincronizarse en todas las vistas

USUARIO: PROVEEDOR DE SERVICIOS
HU08: Gestionar Servicios Ofrecidos
Como proveedor
Quiero administrar los servicios que ofrezco
Para mantener actualizada mi oferta
Criterios de Aceptación:

- Debo poder agregar nuevos servicios con nombre, descripción y precio
- Debo poder editar servicios existentes
- Debo poder desactivar servicios temporalmente
- Debo ver listado de todos mis servicios activos

HU09: Configurar Horario de Trabajo
Como proveedor
Quiero definir mi horario de disponibilidad
Para controlar cuándo aceptar citas
Criterios de Aceptación:

- Debo poder establecer horarios por día de la semana
- Debo poder configurar horas de inicio y fin
- Debo poder definir breaks o horarios no disponibles
- El sistema debe bloquear citas fuera del horario configurado
-

HU10: Visualizar Agenda de Citas
Como proveedor
Quiero ver mi agenda de citas programadas
Para organizar mi trabajo diario
Criterios de Aceptación:

- Debo ver vista diaria/semanal de citas agendadas
- Cada cita debe mostrar cliente, servicio y hora
- Debo poder filtrar por servicio o estado
- Debo recibir notificaciones de nuevas citas

HU11: Confirmar o Rechazar Citas
Como proveedor
Quiero confirmar o rechazar citas solicitadas
Para mantener control sobre mi agenda
Criterios de Aceptación:

- Debo recibir notificación de nuevas solicitudes de cita
- Debo poder confirmar citas dentro de 1 hora de recibidas
- Debo poder rechazar citas con motivo opcional
- El cliente debe recibir notificación de la decisión

HU12: Gestionar Estado de Citas
Como proveedor
Quiero marcar citas como completadas o no asistidas
Para mantener estadísticas precisas
Criterios de Aceptación:

- Debo poder marcar cita como "completada" después del servicio
- Debo poder marcar como "no asistió" si cliente no se presenta
- El sistema debe calcular porcentaje de asistencia
- Debo poder agregar notas internas a cada cita

USUARIO: ADMINISTRADOR
HU13: Gestionar Usuarios del Sistema
Como administrador
Quiero administrar usuarios registrados
Para mantener la seguridad del sistema
Criterios de Aceptación:

- Debo ver listado de todos los usuarios
- Debo poder suspender usuarios problemáticos
- Debo poder ver estadísticas de uso por usuario
- Debo poder restablecer contraseñas si es necesario

HU14: Generar Reportes Estadísticos
Como administrador
Quiero generar reportes de uso del sistema
Para tomar decisiones basadas en datos
Criterios de Aceptación:

- Debo poder generar reportes por período (semanal, mensual)

- Debo ver estadísticas de citas (totales, completadas, canceladas)

Requerimientos Funcionales

Módulo de Autenticación (Sprint 1)

- **RF01:** Registro de usuarios con nombre, correo, teléfono y contraseña.
- **RF02:** Inicio de sesión seguro mediante correo y contraseña.
- **RF03:** Recuperación de contraseña vía correo electrónico.

Módulo de Perfil y Gestión (Sprint 2)

- **RF04:** Edición de perfil de usuario (nombre, teléfono, foto).
- **RF05:** Configuración de horarios de disponibilidad para proveedores.
- **RF06:** Visualización de perfil público para clientes.

Módulo de Servicios (Sprint 2-3)

- **RF07:** Catálogo de servicios ofertados con nombre, descripción y precio.
- **RF08:** Agregar nuevos servicios (opcional para proveedores).
- **RF09:** Eliminar o servicios existentes.

Módulo de Citas (Sprint 3-4)

- **RF10:** Agendar nuevas citas seleccionando servicio, operador (dependiendo del servicio), fecha y hora.
- **RF11:** Visualizar citas vigentes en panel principal.
- **RF12:** Cancelar o reprogramar citas existentes.
- **RF13:** Historial completo de citas anteriores.

Módulo de Calendario (Sprint 4)

- **RF14:** Vista de calendario mensual con citas marcadas.
- **RF15:** Detalle de cita al seleccionar fecha (servicio, proveedor, hora).
- **RF16:** Vista semanal/diaria para gestión detallada de horarios.

Módulo de Notificaciones (Sprint 5)

- **RF17:** Notificaciones directo al móvil para recordatorios de citas.
- **RF18:** Notificaciones de confirmación y cancelación.

Requerimientos No Funcionales

Usabilidad

- **RNF01:** Interfaz intuitiva con tiempo de aprendizaje menor a 10 minutos.
- **RNF02:** Diseño responsive compatible con iOS y Android.

- **RNF03:** Navegación consistente entre secciones (máximo 3 clics para funciones principales).

Fiabilidad

- **RNF04:** Disponibilidad del horario durante horario comercial (Dentro del horario del servicio).

Plan de Desarrollo por Sprints (5 Sprints)

Sprint 1: Base y Autenticación (Primera Entrega)

- Módulo de registro y login.

Sprint 2: Perfiles y Servicios (Segunda Entrega

- Gestión de perfiles de usuario
- Catálogo de servicios
- Configuración de horarios

Sprint 3: Núcleo de Citas (Tercera Entrega)

- Agendado de citas
- Panel de citas vigentes
- Cancelación de citas

Sprint 4: Calendario e Historial (Cuarta Entrega)

- Vista de calendario interactivo
- Historial de citas
- Optimizaciones de UI/UX

Sprint 5: Notificaciones y Pulido (Quinta entrega)

- Sistema de notificaciones
- Testing integral

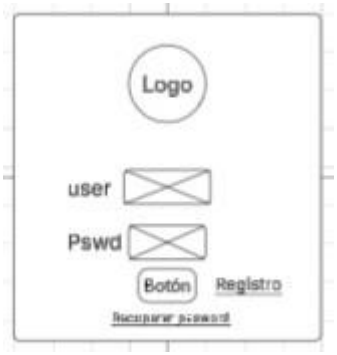
Calendario

En la siguiente imagen, se establecen físicamente las fechas de entrega de cada sprint, con los días establecidos para trabajar.

Pantallas.

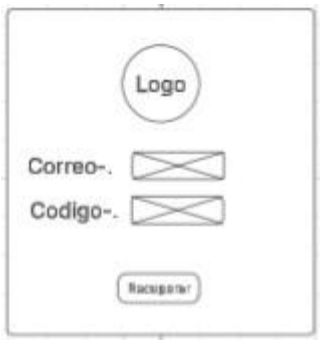
Inicio y registro de sesión

Log pricipal

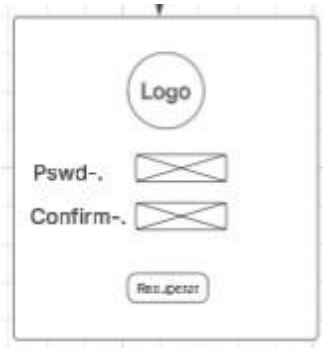


Creación de cuenta

Recuperacion

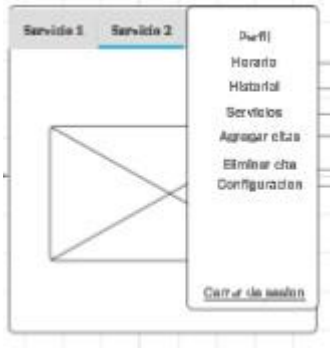


Cambio de contraseña

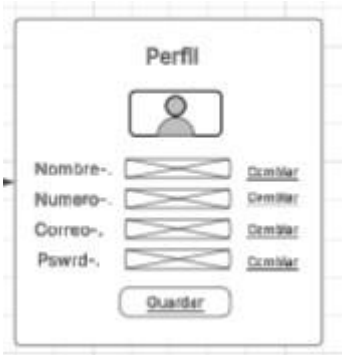


Pantalla inicial

Menú



Perfil



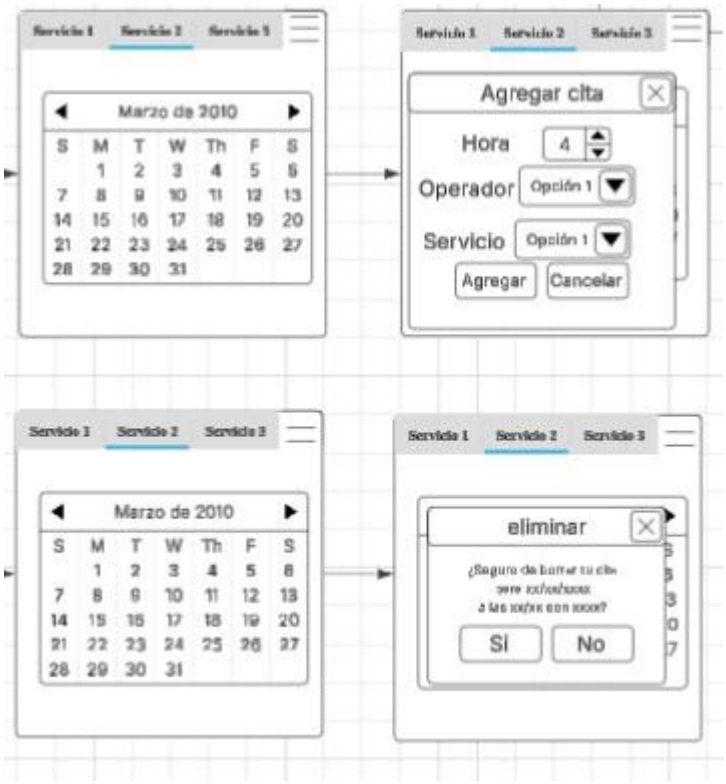
Horario

Historial de citas

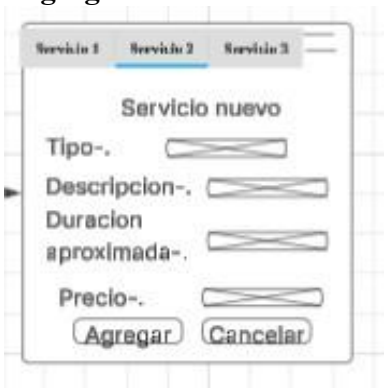
Servicios

Búsqueda y agregar servicios

Agregar y eliminar citas



Agregar servicio-Proveedor



Status de cita

Primer Sprint.

En la presentación del primer sprint se hizo la entrega de la aplicación que es desarrollada en Flutter, la cual esta implementada en la arquitectura MVVM (modelo vista, vista modelo). Esta arquitectura consiste en separar la interfaz del usuario de la lógica de negocio, esto es donde el modelo son los datos y la lógica de negocio, en la vista esta la interfaz visible para el usuario, y en la vista modelo actúa como un intermediario que contiene la lógica del usuario, además de sincronizar los datos entre el modelo y la vista.

La fecha establecida en el calendario es del 10 del octubre del 2025 para la entrega del primer sprint, la cual se tiene establecido que en dicha fecha mencionada se entregaran los siguientes datos:

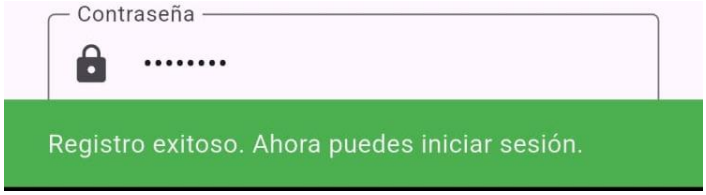
- **Sprint 1:** Base y Autenticación:
  - ✓ Módulo de registro y login.

El objetivo de este sprint es el de establecer la base de la aplicación e implementar el prototipo de autenticación de usuarios.

- ✓ **Módulo de registro:**

En este módulo se observan las funcionalidades que permiten a los usuarios crear una nueva cuenta en la aplicación, en esta pantalla podemos en observar el primer contacto entre el usuario y el sistema.

Al llenar los datos que se presenta la pantalla, se mostrara una notificación en la parte de abajo, de color verde, en la cual te notificara que el usuario fue registrado con éxito.



- ✓ **Login**

En este apartado podemos encontrar el proceso por el cual el usuario va acceder a la aplicación de una manera segura, en el login, se solicitará el correo y la contraseña con el que el usuario se registró, al momento de dar inicio de sesión se mostrará una notificación en la parte de abajo que dirá, el ID del usuario ingresado.

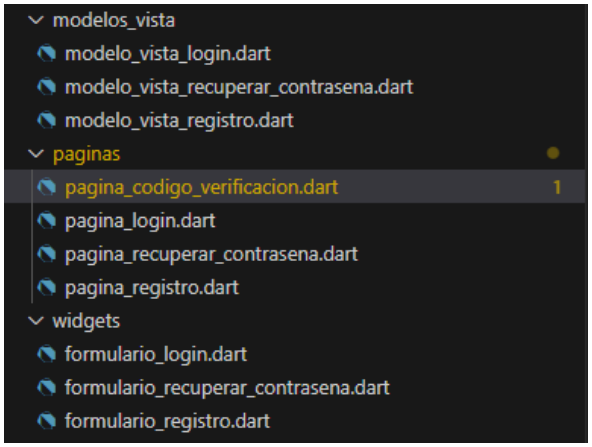
- ✓ **Recuperación de contraseña**

Al momento que el usuario no recuerde su contraseña, pude recuperarla, al dar clic en el apartado de ¿olvidaste tu contraseña?, te mandara una pantalla, la cual te pedirá tu correo para poder enviarte un código que es el que ayudara al usuario a registrar una nueva contraseña.

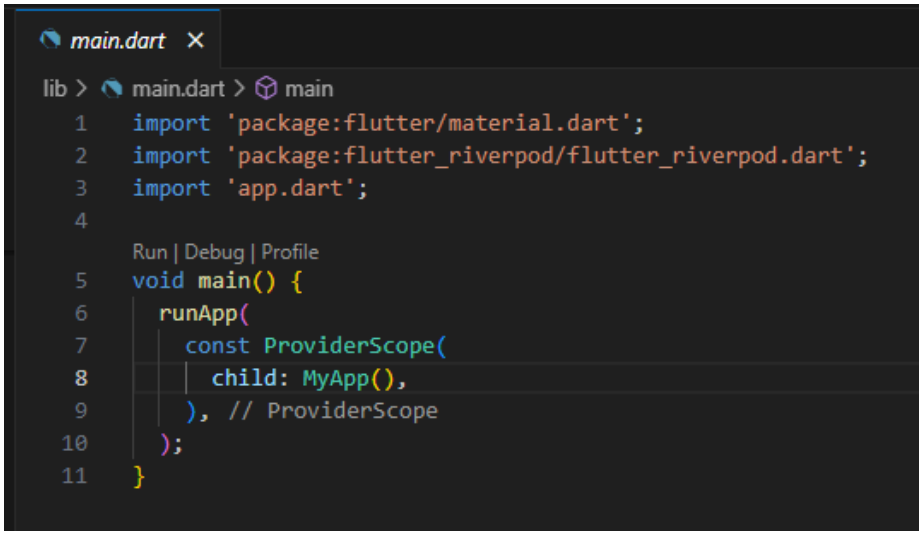
Presentación del tipo de arquitectura

La aplicación cuenta con la arquitectura modelo vista, vista modelo (MVVM), la cual tiene como objetivo el separar la vista, que es la interfaz de usuario del modelo, que se conoce como la lógica de negocio, pero esto se hace mediante una capa intermedia (vista modelo).

Como se había mencionado, en el primer sprint se hizo la entrega del módulo de registro, lo realizamos en la siguiente parte del código, las siguientes carpetas se observarán las entregas mencionadas



En el main únicamente mandamos a llamar nuestra app



```
main.dart ×
lib > main.dart > main
1   import 'package:flutter/material.dart';
2   import 'package:flutter_riverpod/flutter_riverpod.dart';
3   import 'app.dart';
4
    Run | Debug | Profile
5   void main() {
6     runApp(
7       const ProviderScope(
8         child: MyApp(),
9       ), // ProviderScope
10    );
11  }
```

```dart
app.dart
lib > app.dart > MyApp
import 'Caracteristicas/Autentificacion/presentacion/paginas/pagina_login.

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Citas Multi-Servicios',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: true,
      ), // ThemeData
      home: const PaginaLogin(),
      debugShowCheckedModeBanner: false,
    ); // MaterialApp
  }
}
```

En la configuración de nuestra app creamos las bases de nuestra app definiendo el tirulo del proyecto y el modelo estándar con el useMaterial3 asignando un modelo predeterminado

```dart
validadores.dart
lib > Recurso C-Corazon > utilidades > validadores.dart > Validadores
class Validadores {
  static String? validarCorreo(String? valor) {
    if (valor == null || valor.isEmpty) {
      return 'El correo es requerido';
    }

    final regex = RegExp(r'^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$');
    if (!regex.hasMatch(valor)) {
      return 'Ingrese un correo válido';
    }

    return null;
  }

  static String? validarContrasena(String? valor) {
    if (valor == null || valor.isEmpty) {
      return 'La contraseña es requerida';
    }

    if (valor.length < 6) {
      return 'La contraseña debe tener al menos 6 caracteres';
    }

    return null;
  }

  static String? validarConfirmarContrasena(String? valor, String contrase
    if (valor == null || valor.isEmpty) {
      return 'Confirme la contraseña';
    }

```

Aquí en la clase validación asignamos Validadores que utilizaremos alrededor del código para rectificar que las validaciones sean correctas a través de estas funciones predeterminadas que nos van a permitir reutilizarlas a lo largo de los códigos

```dart
usuario.dart
lib > Caracteristicas > Autentificacion > dominio > entidades > usuario.dart > Us

class Usuario {
  final String id;
  final String nombre;
  final String correo;
  final String telefono;

  Usuario({
    required this.id,
    required this.nombre,
    required this.correo,
    required this.telefono,
    required String contrasena,
  });

  // Método para crear un usuario desde un mapa (JSON)
  factory Usuario.desdeMap(Map<String, dynamic> mapa) {
    return Usuario(
      id: mapa['id'] ?? '',
      nombre: mapa['nombre'] ?? '',
      correo: mapa['correo'] ?? '',
      telefono: mapa['telefono'] ?? '',
      contrasena: mapa['contrasena'] ?? '',
    );
  }

  // Convertir usuario a mapa para enviar a API
  Map<String, dynamic> toMap() {
    return {
      'id': id,
```

En esta parte de código definimos la entidad de los recursos utilizados en Usurario y lo modelamos para usarlo y preparo mapeándolo para poder usarlo también en el método JSON y después poder utilizarlo en las llamadas de los servicios

Después de eso vamos revisando la implementación de las pantallas de conforme a las arquitecturas

caso_uso_login.dart

lib > Caracteristicas > Autentificacion > dominio > casos_uso > caso_uso_login.dart > CasoUsoLog

```dart
import '../entidades/usuario.dart';
import '../repositorios/repositorio_autenticacion.dart';

class CasoUsoLogin {
  final RepositorioAutenticacion repositorio;

  CasoUsoLogin(this.repositorio);

  Future<Usuario> ejecutar(String correo, String contrasena) async {
    // Aquí se pueden agregar validaciones adicionales
    if (correo.isEmpty || contrasena.isEmpty) {
      throw Exception('Correo y contraseña son requeridos');
    }

    return await repositorio.login(correo, contrasena);
  }
}
```

En esta parte del cogido lo que se ase es rectificar los datos para poder hacer la el login la lógica de negocios

pagina_login.dart

lib > Caracteristicas > Autentificacion > presentacion > paginas > pagina_login.dart > PaginaLogin

```dart
class PaginaLogin extends ConsumerWidget {
  const PaginaLogin({super.key});

  void _navegarARegistro(BuildContext context) {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const PaginaRegistro()),
    );
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final estado = ref.watch(modeloVistaLoginProvider);
    final modeloVista = ref.read(modeloVistaLoginProvider.notifier);

    // Si el login fue exitoso, navegar al dashboard
    if (estado.usuario != null) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text('Bienvenido ${estado.usuario!.nombre}'),
            backgroundColor: Colors.green,
          ), // SnackBar
        );
        // Aquí navegaríamos al dashboard en el siguiente sprint
      });
    }

    return Scaffold(
      body: SafeArea(
        child: SingleChildScrollView(
```

Aquí definimos las vistas tanto del formulario como de la página del login lo seccionamos para tener más control de la estructura

formulario_login.dart

lib > Caracteristicas > Autentificacion > presentacion > widgets > formulario_login.

```dart
class FormularioLogin extends StatefulWidget {
  const FormularioLogin({
    this.error,
  });

  @override
  State<FormularioLogin> createState() => _FormularioLogin
}

class _FormularioLoginState extends State<FormularioLogin>
  final _formKey = GlobalKey<FormState>();
  final _controladorCorreo = TextEditingController();
  final _controladorContrasena = TextEditingController();

  @override
  void dispose() {
    _controladorCorreo.dispose();
    _controladorContrasena.dispose();
    super.dispose();
  }

  void _enviarFormulario() {
    if (_formKey.currentState!.validate()) {
      widget.alIniciarSesion(
        _controladorCorreo.text.trim(),
        _controladorContrasena.text,
      );
    }
  }

  @override
  Widget build(BuildContext context) {
```

Y aquí relacionamos la vista con el modelo a través de la sección view model

modelo_vista_login.dart

lib > Caracteristicas > Autentificacion > presentacion > modelos_vista > modelo_vista_login.dart >

```dart
// ViewModel usando StateNotifier
class ModeloVistaLogin extends StateNotifier<EstadoLogin> {
  final CasoUsoLogin casoUsoLogin;

  ModeloVistaLogin(this.casoUsoLogin) : super(const EstadoLogin());

  Future<void> iniciarSesion(String correo, String contrasena) async {
    // Estado de carga
    state = state.copiarCon(cargando: true, error: null);

    try {
      final usuario = await casoUsoLogin.ejecutar(correo, contrasena);
      state = state.copiarCon(cargando: false, usuario: usuario);
    } catch (e) {
      state = state.copiarCon(cargando: false, error: e.toString());
    }
  }

  void limpiarError() {
    state = state.copiarCon(error: null);
  }
}
```

De la misma manera que con el login generamos dentro del caso de uso la lógica del negocio del registro

caso_uso_registro.dart

lib > Caracteristicas > Autentificacion > dominio > casos_uso > caso_uso_registro.dart > CasoUsoRe

```dart
// Caso de uso para el proceso de registro
import '../entidades/usuario.dart';
import '../repositorios/repositorio_autenticacion.dart';

class CasoUsoRegistro {
  final RepositorioAutenticacion repositorio;

  CasoUsoRegistro(this.repositorio);

  Future<Usuario> ejecutar(Usuario usuario, String contrasena) async {
    // Validaciones básicas
    if (usuario.correo.isEmpty || contrasena.isEmpty) {
      throw Exception('Datos incompletos');
    }

    return await repositorio.registrar(usuario, contrasena);
  }
}
```

Aquí definimos las vistas tanto del formulario como de la página del Registro lo seccionamos para tener más control de la estructura

pagina_registro.dart

lib > Caracteristicas > Autentificacion > presentacion > paginas > pagina_registro.dart > casoUsoRegis

```dart
class PaginaRegistro extends ConsumerWidget {
  const PaginaRegistro({super.key});

  void _regresarALogin(BuildContext context) {
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final estado = ref.watch(modeloVistaRegistroProvider);
    final modeloVista = ref.read(modeloVistaRegistroProvider.notifier);

    // Si el registro fue exitoso, regresar al login
    if (estado.usuario != null) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text('Registro exitoso. Ahora puedes iniciar sesión.'
            backgroundColor: Colors.green,
          ), // SnackBar
        );
        Navigator.pop(context);
      });
    }
```

formulario_registro.dart

lib > Caracteristicas > Autentificacion > presentacion > widgets > formulario_registro.dart > ...

```dart
class _FormularioRegistroState extends State<FormularioRegistro> {
  final _formKey = GlobalKey<FormState>();
  final _controladorNombre = TextEditingController();
  final _controladorCorreo = TextEditingController();
  final _controladorTelefono = TextEditingController();
  final _controladorContrasena = TextEditingController();
  final _controladorConfirmarContrasena = TextEditingController();

  @override
  void dispose() {
    _controladorNombre.dispose();
    _controladorCorreo.dispose();
    _controladorTelefono.dispose();
    _controladorContrasena.dispose();
    _controladorConfirmarContrasena.dispose();
    super.dispose();
  }

  void _enviarFormulario() {
    if (_formKey.currentState!.validate()) {
      widget.alRegistrar(
        _controladorNombre.text.trim(),
        _controladorCorreo.text.trim(),
        _controladorTelefono.text.trim(),
        _controladorContrasena.text,
      );
    }
  }
```

Y aquí relacionamos la vista con el modelo a través de la sección view model

modelo_vista_registro.dart

lib > Caracteristicas > Autentificacion > presentacion > modelos_vista > modelo_vista_registro.dart > M

```dart
class ModeloVistaRegistro extends StateNotifier<EstadoRegistro> {
  ModeloVistaRegistro(this.casoUsoRegistro) : super(const EstadoRegistro()

  Future<void> registrarUsuario(
    String nombre,
    String correo,
    String telefono,
    String contrasena,
  ) async {
    state = state.copiarCon(cargando: true, error: null);

    try {
      final usuario = Usuario(
        id: '', // Se generará en el repositorio
        nombre: nombre,
        correo: correo,
        telefono: telefono,
        contrasena: contrasena,
      );

      final usuarioRegistrado = await casoUsoRegistro.ejecutar(usuario, co
      state = state.copiarCon(cargando: false, usuario: usuarioRegistrado)
    } catch (e) {
      state = state.copiarCon(cargando: false, error: e.toString());
    }
  }
```

caso_uso_recuperar_contrasena.dart

risticas > Autentificacion > dominio > casos_uso > caso_uso_recuperar_contrasena.dart > CasoUsoRecup

```dart
class CasoUsoRecuperarContrasena {
  final RepositorioAutenticacion repositorio;

  CasoUsoRecuperarContrasena(this.repositorio);

  // Solicita el envío de código de verificación al correo
  Future<void> solicitarRecuperacion(String correo) async {
    if (correo.isEmpty) {
      throw Exception('El correo electrónico es requerido');
    }

    // Validar formato de correo
    final regex = RegExp(r'^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$');
    if (!regex.hasMatch(correo)) {···
    }

    await repositorio.solicitarRecuperacionContrasena(correo);
  }

  // Verifica el código de verificación
  Future<void> verificarCodigo(String correo, String codigo) async {
    if (codigo.isEmpty || codigo.length != 6) {
      throw Exception('El código debe tener 6 dígitos');
    }

    await repositorio.verificarCodigoRecuperacion(correo, codigo);
  }

  // Establece nueva contraseña después de verificación
  Future<void> establecerNuevaContrasena(String correo, String codigo, Str
    if (nuevaContrasena.length < 6) {
```

de la misma manera que con el login y el registro generamos dentro del caso de uso la lógica del negocio del recuperar contraseña

pagina_recuperar_contrasena.dart

lib > Caracteristicas > Autentificacion > presentacion > paginas > pagina_recuperar_contrasena.dart >

```dart
class PaginaRecuperarContrasena extends ConsumerWidget {
  const PaginaRecuperarContrasena({super.key});

  void _navegarACodigoVerificacion(BuildContext context, String correo) {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => PaginaCodigoVerificacion(correo: correo),
      ), // MaterialPageRoute
    );
  }

  void _regresarALogin(BuildContext context) {
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final estado = ref.watch(proveedorModeloVistaRecuperarContrasena);
    final modeloVista = ref.read(proveedorModeloVistaRecuperarContrasena.n

    // Navegar automáticamente a la pantalla de código cuando se envíe el
    if (estado.codigoEnviado && estado.correo != null) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        _navegarACodigoVerificacion(context, estado.correo!);
      });
    }

    return Scaffold(
      appBar: AppBar(
        leading: IconButton(
```

Aquí definimos las vistas tanto del formulario como de la página del Recuperar contraseña lo seccionamos para tener más control de la estructura

formulario_recuperar_contrasena.dart

lib > Caracteristicas > Autentificacion > presentacion > widgets > formulario_recuperar_contrasena.dart >

```dart
class _FormularioRecuperarContrasenaState extends State<FormularioRecuperar
  final _formKey = GlobalKey<FormState>();
  final _controladorCorreo = TextEditingController();

  @override
  void dispose() {
    _controladorCorreo.dispose();
    super.dispose();
  }

  void _enviarFormulario() {
    if (_formKey.currentState!.validate()) {
      widget.alSolicitarRecuperacion(_controladorCorreo.text.trim());
    }
  }

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: Column(
        children: [
          // Mensaje informativo
          const Text(
            'Ingresa tu correo electrónico y te enviaremos un código de ve
            textAlign: TextAlign.center,
            style: TextStyle(
              fontSize: 16,
```

modelo_vista_recuperar_contrasena.dart

lib > Caracteristicas > Autentificacion > presentacion > modelos_vista > modelo_vista_recuperar_contrase

```dart
// ViewModel para el proceso de recuperación de contraseña
class ModeloVistaRecuperarContrasena extends StateNotifier<EstadoRecuperac
  final CasoUsoRecuperarContrasena casoUsoRecuperarContrasena;

  ModeloVistaRecuperarContrasena(this.casoUsoRecuperarContrasena)
      : super(const EstadoRecuperacionContrasena());

  // Paso 1: Solicitar envío de código de verificación
  Future<void> solicitarCodigoVerificacion(String correo) async {
    state = state.copiarCon(cargando: true, error: null);

    try {
      await casoUsoRecuperarContrasena.solicitarRecuperacion(correo);
      state = state.copiarCon(
        cargando: false,
        correo: correo,
        codigoEnviado: true,
        error: null,
      );
    } catch (e) {
      state = state.copiarCon(
        cargando: false,
        error: e.toString(),
      );
    }
  }

  // Paso 2: Verificar código recibido
```

Y aquí relacionamos la vista con el modelo a través de la sección view model

Notas del Primer Sprint

El día de la presentación del primer sprint se hicieron algunos comentarios los cuales se tomaron en cuenta para la presentación de este nuevo sprint, los cuales son:

- ✓ Cambio de los botones, deben ser un poco más circulares.
- ✓ Cambio de Arquitectura, ya que no era la indicada.

Segundo Sprint.

En la presentación del segundo sprint se hizo la entrega de los avances de la aplicación, la fecha establecida en el calendario es del 24 de octubre del 2025, para esta entrega del segundo sprint, se estableció que se entregarían los siguientes datos:

- **Sprint 2: Perfiles y Servicios:**

✓ Gestión de perfiles de usuario.

Se integrará la creación, edición y visualización de perfiles, cada uno de los usuarios podrá registrar sus datos personales, tomando en cuenta los siguientes datos: nombre, edad y foto.

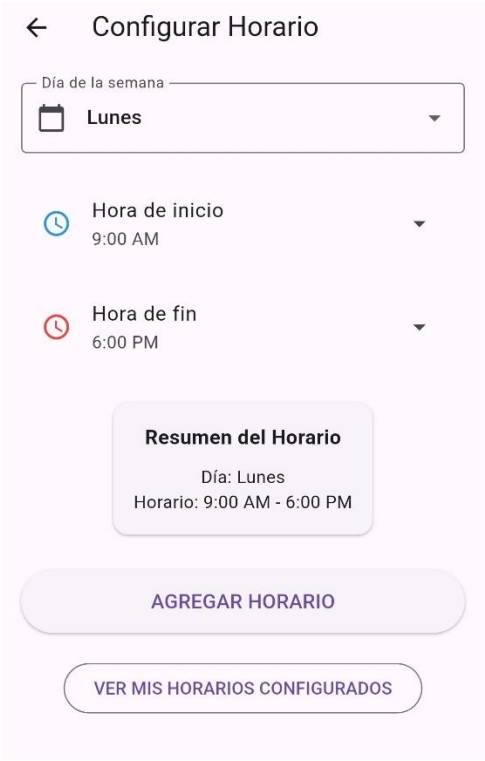
✓ Catálogo de servicios

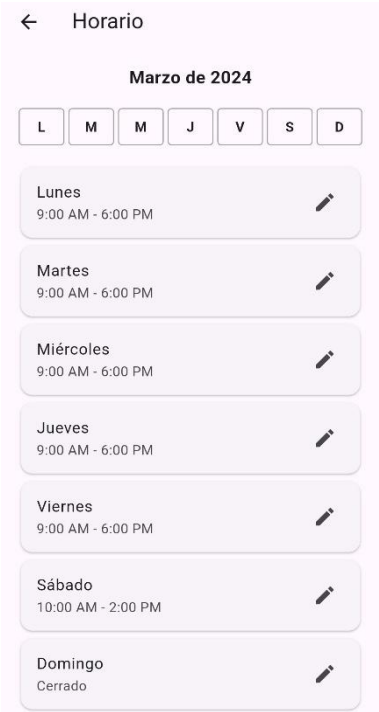
Se permitirá tener visualizar los diferentes servicios que ofrece la aplicación, algunos servicios que se podrán observar a futuro, será una Barberia, dental, etc. Esta parte de la se podrán mostrar (clientes) y gestionar los catálogos. (proveedor)

Cliente: Se podrán mostrar los servicios disponibles para el usuario.

Proveedor: Se podrá gestionar los servicios, agregar o eliminar servicios.

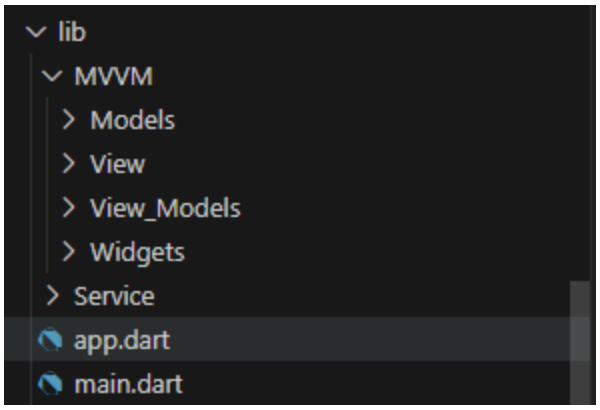
- ✓ Configuración de horarios: El usuario podrá definir la disponibilidad o preferencia de horario para sus citas.

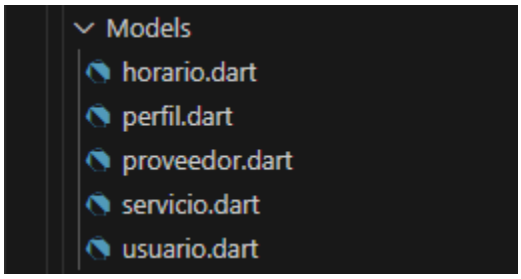




Código del 2do Sprint

En la segunda semana de entrega se desarrolló el código del segundo sprint, el objetivo con el que se trabajo fue poder establecer la interacción de los usuarios y los servicios, esta entrega se puede observar en las siguientes carpetas mencionas.





Cada una de las carpetas cuenta con una función, en la carpeta de models, podemos encontrar las clases de horario, perfil, proveedor, servicio y usuario.

En la clase horario podemos encontrar que se definen las características de los horarios con que se trabaja el servicio.

horario.dart × usuario.dart

lib > MVVM > Models > horario.dart > Horario

```dart
class Horario {
  final String id;
  final String proveedorId;
  final String diaSemana;
  final String horaInicio;
  final String horaFin;
  final bool activo;

  Horario({
    required this.id,
    required this.proveedorId,
    required this.diaSemana,
    required this.horaInicio,
    required this.horaFin,
    this.activo = true,
  });

  factory Horario.desdeMap(Map<String, dynamic> mapa) {
    return Horario(
      id: mapa['id'] ?? '',
      proveedorId: mapa['proveedorId'] ?? '',
      diaSemana: mapa['diaSemana'] ?? '',
      horaInicio: mapa['horaInicio'] ?? '',
      horaFin: mapa['horaFin'] ?? '',
      activo: mapa['activo'] ?? true,
    );
  }

  Map<String, dynamic> toMap() {
    return {
      'proveedorId': proveedorId,
      'diaSemana': diaSemana,
      'horaInicio': horaInicio,
      'horaFin': horaFin,
      'activo': activo,
    };
  }

  String get horarioCompleto => '$horaInicio - $horaFin';
```

En la clase perfil definimos los datos esenciales para identificar al usuario.

horario.dart perfil.dart ×

lib > MVVM > Models > perfil.dart > Perfil > toMap

```dart
class Perfil {
  final String usuarioId;
  final String nombre;
  final String telefono;
  final String correo;
  final String tipoUsuario;

  Perfil({
    required this.usuarioId,
    required this.nombre,
    required this.telefono,
    required this.correo,
    required this.tipoUsuario,
  });

  Map<String, dynamic> toMap() {
    return {
      'usuarioId': usuarioId,
      'nombre': nombre,
      'telefono': telefono,
      'correo': correo,
      'tipoUsuario': tipoUsuario,
    };
  }
}
```

```dart
proveedor.dart
lib > MVVM > Models > proveedor.dart > Proveedor > Proveedor
class Proveedor extends Usuario {
  final List<String>? serviciosOfrecidos;
  final List<String>? horariosAtencion;
  final bool? verificado;
  final String? fotoNegocio;
  final String? telefonoNegocio;
  final String? sitioWeb;
  final Map<String, dynamic>? configuracion;

  Proveedor({
    required String id,
    required String nombre,
    required String correo,
    required String telefono,
    this.nombreNegocio,
    this.descripcionNegocio,
    this.direccion,
    this.calificacion,
    this.totalCalificaciones,
    this.serviciosOfrecidos,
    this.horariosAtencion,
    this.verificado = false,
    this.fotoNegocio,
    this.telefonoNegocio,
    this.sitioWeb,
    this.configuracion,
    required String pasword,
  }) : super(
          id: id,
          nombre: nombre,
          correo: correo,
          telefono: telefono,
          tipoUsuario: 'proveedor',
          pasword: pasword,
        );
```

En la clase proveedor podemos observar que se gestiona la información, de la identidad, además de la lógica de la reputación de usuario que provee los servicios dentro de la aplicación.

En la clase servicio se estrcutura toda informacion de un producto o servicio especifico que ofrece el proveedor.

```dart
servicio.dart
lib > MVVM > Models > servicio.dart > Servicio > Servicio
class Servicio {

  Servicio({
    required this.id,
    required this.proveedorId,
    required this.nombre,
    required this.descripcion,
    required this.duracion,
    required this.precio,
    required this.categoria,
    this.activo = true,
    this.nombreProveedor,
  });

  factory Servicio.desdeMap(Map<String, dynamic> mapa) {
    return Servicio(
      id: mapa['id'] ?? '',
      proveedorId: mapa['proveedorId'] ?? '',
      nombre: mapa['nombre'] ?? '',
      descripcion: mapa['descripcion'] ?? '',
      duracion: mapa['duracion'] ?? 0,
      precio: (mapa['precio'] ?? 0.0).toDouble(),
      categoria: mapa['categoria'] ?? '',
      activo: mapa['activo'] ?? true,
      nombreProveedor: mapa['nombreProveedor'],
    );
  }
```

```dart
usuario.dart
lib > MVVM > Models > usuario.dart > Usuario
class Usuario {
  Usuario({
    required this.pasword,
  });

  factory Usuario.desdeMap(Map<String, dynamic> mapa) {
    return Usuario(
      id: mapa['id'] ?? '',
      nombre: mapa['nombre'] ?? '',
      correo: mapa['correo'] ?? '',
      telefono: mapa['telefono'] ?? '',
      tipoUsuario: mapa['tipoUsuario'] ?? 'cliente',
      pasword: mapa['pasword'] ?? '',
    );
  }

  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'nombre': nombre,
      'correo': correo,
      'telefono': telefono,
      'tipoUsuario': tipoUsuario,
    };
  }
```

En la clase usuario se estructura los tipos de usuarios como los datos de registro que se presentaran en la aplicación.

En la carpeta view, se tiene la interfaz de usuario, aquí se ubicaran las pantallas donde va interactuar el usuario.

```dart
void _agregarHorario(BuildContext context) {
  if (_formKey.currentState!.validate()) {
    final authService = Provider.of<AuthService>(context, listen: false);
    final proveedorId = authService.usuarioActual!.id;

    final nuevoHorario = Horario(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      proveedorId: proveedorId,
      diaSemana: _diaSeleccionado,
      horaInicio: '${_horaInicio.hour.toString().padLeft(2, '0')}:${_horaInicio.minute.toString().padLeft(2, '0')}',
      horaFin: '${_horaFin.hour.toString().padLeft(2, '0')}:${_horaFin.minute.toString().padLeft(2, '0')}',
      activo: true,
    ); // Horario

    final viewModel = Provider.of<HorarioViewModel>(context, listen: false);
    viewModel.agregarHorario(nuevoHorario).then((_) {
      if (viewModel.error == null) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text('Horario agregado correctamente'),
            backgroundColor: Colors.green,
          ), // SnackBar
```

En esta clase, es para agregar el horario que proveedor eliga.

En la clase agregar servicio, saldra la pantalla donde el proveedor podra dar de alta el servicio necesario

```dart
agregar_servicio.dart
lib > MVVM > View > agregar_servicio.dart > _PaginaAgregarServicioState > _agregarServicio
class _PaginaAgregarServicioState extends State<PaginaAgregarServicio> {

  void _agregarServicio() {
    if (_formKey.currentState!.validate()) {
      final authService = context.read<AuthService>();

      final nuevoServicio = Servicio(
        id: '', // El ID se genera en el servicio
        proveedorId: authService.usuarioActual!.id,
        nombre: _controladorNombre.text.trim(),
        descripcion: _controladorDescripcion.text.trim(),
        duracion: int.tryParse(_controladorDuracion.text) ?? 30,
        precio: double.tryParse(_controladorPrecio.text) ?? 0.0,
        categoria: _categoriaSeleccionada,
        nombreProveedor: authService.usuarioActual!.nombre,
      );

      final viewModel = context.read<ServiciosViewModel>();
      viewModel.agregarServicio(nuevoServicio).then((_) {
        if (mounted && viewModel.error == null) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Servicio agregado con éxito'), backgroundColor: Colors.green),
          );
          Navigator.pop(context);
        }
      });
    }
  }
```

```dart
  @override
  Widget build(BuildContext context) {
    final authService = Provider.of<AuthService>(context, listen: false);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Panel Administrador'),
        backgroundColor: Colors.blue,
        actions: [
          IconButton(
            icon: const Icon(Icons.logout),
            onPressed: () {
              authService.cerrarSesion();
            },
          ), // IconButton
        ],
      ), // AppBar
      drawer: Drawer(
        child: ListView(
          padding: EdgeInsets.zero,
          children: [
            UserAccountsDrawerHeader(
              accountName: Text(usuario.nombre),
              accountEmail: Text(usuario.correo),
              decoration: const BoxDecoration(color: Colors.blue),
              currentAccountPicture: const CircleAvatar(
                backgroundColor: Colors.white,
                child: Icon(Icons.admin_panel_settings, color: Colors.blue, size: 40),
              ), // CircleAvatar
```

En esta clase se muesta la pantalla para el rol de administrado y que muestre las herramientas de gestión y estadísticas.

En esta clase, la pantalla principal que ve un cliente, aquí se muestran los servicios.

```dart
dashboard_cliente.dart
lib > MVVM > View > dashboard_cliente.dart > DashboardCliente > build
class DashboardCliente extends StatelessWidget {
  Widget build(BuildContext context) {
        ],
      ), // AppBar
      drawer: Drawer(
        child: ListView(
          children: [
            UserAccountsDrawerHeader(
              accountName: Text(usuario.nombre),
              accountEmail: Text(usuario.correo),
              currentAccountPicture: const CircleAvatar(
                backgroundColor: Colors.white,
                child: Icon(Icons.person, color: Colors.blue),
              ), // CircleAvatar
            ), // UserAccountsDrawerHeader
            ListTile(
              leading: const Icon(Icons.home),
              title: const Text('Inicio'),
              onTap: () => Navigator.pop(context),
            ), // ListTile
            ListTile(
              leading: const Icon(Icons.work),
              title: const Text('Buscar Servicios'),
              onTap: () {
                Navigator.pop(context);
                Navigator.push(context, MaterialPageRoute(builder: (context) => const PaginaS
              },
            ), // ListTile
```

En esta clase se muestra la pantalla principal que ve el proveedor que se muestran las citas, calificaciones y acceso a gestión de servicios y horarios.

```
dashboard_cliente.dart   dashboard_proveedor.dart ×
lib > MVVM > View > dashboard_proveedor.dart > ...
11  class DashboardProveedor extends StatelessWidget {
17    Widget build(BuildContext context) {
34    }
35
36    Widget _buildBody(BuildContext context) {
37      return Center(
38        child: SingleChildScrollView(
39          padding: const EdgeInsets.all(32.0),
40          child: Column(
41            mainAxisAlignment: MainAxisAlignment.center,
42            children: [
43              Icon(Icons.business_center, size: 80, color: Colors.blue[700]),
44              const SizedBox(height: 20),
45              Text(
46                '¡Bienvenido, ${usuario.nombre}!',
47                style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
48                textAlign: TextAlign.center,
49              ), // Text
50              const SizedBox(height: 10),
51              const Text(
52                'Panel de Control - Proveedor',
53                style: TextStyle(fontSize: 16, color: Colors.grey),
54                textAlign: TextAlign.center,
55              ), // Text
56              const SizedBox(height: 40),
57              _buildActionButton(
58                context: context,
59                icon: Icons.add_business,
60                label: 'Agregar Nuevo Servicio',
61                onPressed: () {
62                  Navigator.push(context, MaterialPageRoute(builder: (_) => const PaginaAgregar
```

```
pagina_horario.dart ×
lib > MVVM > View > pagina_horario.dart > ...
3   class PaginaHorario extends StatelessWidget {
7     Widget build(BuildContext context) {
22            Padding(
23              padding: EdgeInsets.symmetric(horizontal: 16),
24              child: Row(
25                mainAxisAlignment: MainAxisAlignment.spaceBetween,
26                children: ['L', 'M', 'M', 'J', 'V', 'S', 'D']
27                    .map((dia) => Expanded(
28                      child: Container(
29                        margin: EdgeInsets.all(2),
30                        padding: EdgeInsets.all(8),
31                        decoration: BoxDecoration(
32                          border: Border.all(color: Colors.grey),
33                          borderRadius: BorderRadius.circular(4),
34                        ), // BoxDecoration
35                        child: Text(
36                          dia,
37                          textAlign: TextAlign.center,
38                          style: TextStyle(fontWeight: FontWeight.bold),
39                        ), // Text
40                      ), // Container
41                    )) // Expanded
42                    .toList(),
43              ), // Row
44            ), // Padding
```

En la clase agregar horario ahí desarrollara la pantalla para que el proveedor pueda añadir o modificar el horario.

En esta clase la pantalla tiene como objetivo el poder ver y editar la información del usuario.

```
Widget _buildConUsuario(BuildContext context, Usuario usuario) {
  return Consumer<PerfilViewModel>(
    builder: (context, viewModel, child) {
      return SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [

            Card(
              child: Padding(
                padding: const EdgeInsets.all(16),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    const Text(
                      'Editar Información',
                      style: TextStyle(
                        fontSize: 18,
                        fontWeight: FontWeight.bold,
                      ), // TextStyle
                    ), // Text
                    const SizedBox(height: 16),
```

```
}

void _eliminarServicio(BuildContext context, String servicioId) {
  showDialog(
    context: context,
    builder: (BuildContext dialogContext) {
      return AlertDialog(
        title: const Text('Confirmar Eliminación'),
        content: const Text('¿Estás seguro de que quieres eliminar este servicio? Esta acci
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext),
            child: const Text('Cancelar'),
          ), // TextButton
          TextButton(
            onPressed: () {
              Navigator.pop(dialogContext);
              final viewModel = Provider.of<ServiciosViewModel>(context, listen: false);
              final proveedorId = Provider.of<AuthService>(context, listen: false).usuarioA

              viewModel.eliminarServicio(servicioId).then((_) {
                // Después de eliminar, recargamos la lista de servicios del proveedor.
                viewModel.cargarServiciosProveedor(proveedorId);
              });
            },
            child: const Text('Eliminar', style: TextStyle(color: Colors.red)),
          ), // TextButton
```

En esta clase se podrán eliminar los servicios.

En esta clase se maneja la validación de los datos de entrada y llama al servicio para crear el nuevo usuario o Proveedor

registro_view_model.dart

lib > MVVM > View_Models > registro_view_model.dart > ...

```dart
import 'package:flutter/material.dart';
import '../Models/usuario.dart';
import '../../Service/auth_service.dart';

class RegistroViewModel with ChangeNotifier {
  final AuthService _authService;

  Usuario? _usuario;
  bool _cargando = false;
  String? _error;

  RegistroViewModel(this._authService);

  Usuario? get usuario => _usuario;
  bool get cargando => _cargando;
  String? get error => _error;

  Future<void> registrarUsuario(String nombre, String correo, String telefono, String contrasena) async {
    _cargando = true;
    _error = null;
    notifyListeners();

    try {
      final nuevoUsuario = Usuario(
        id: '',
        nombre: nombre,
        correo: correo,
        telefono: telefono,
        tipoUsuario: 'cliente',
        pasword: '',
      );
```

horario_view_model.dart

lib > MVVM > View_Models > horario_view_model.dart > ...

```dart
import 'package:flutter/material.dart';
import 'package:flutter_application_appdate2/MVVM/Models/horario.dart';
import 'package:flutter_application_appdate2/Service/horario_service.dart';

class HorarioViewModel with ChangeNotifier {
  final HorarioService _horarioService;

  List<Horario> _horarios = [];
  bool _cargando = false;
  String? _error;

  HorarioViewModel(this._horarioService);

  List<Horario> get horarios => _horarios;
  bool get cargando => _cargando;
  String? get error => _error;

  Future<void> cargarHorariosProveedor(String proveedorId) async {
    _cargando = true;
    _error = null;
    notifyListeners();

    try {
      _horarios = await _horarioService.obtenerHorariosPorProveedor(proveedorId);
    } catch (e) {
      _error = e.toString();
    } finally {
      _cargando = false;
      notifyListeners();
    }
  }
```

En esta clase se gestiona la lógica con la que se hace la configuración de horarios para un proveedor.

En esta clase se gestiona la lógica del Catálogo de Servicios.

servicios_view_model.dart

lib > MVVM > View_Models > servicios_view_model.dart > ...

```dart
import 'package:flutter/material.dart';
import 'package:flutter_application_appdate2/MVVM/Models/servicio.dart';
import 'package:flutter_application_appdate2/Service/servicios_service.dart';

class ServiciosViewModel with ChangeNotifier {
  final ServiciosService _serviciosService;

  List<Servicio> _servicios = [];
  bool _cargando = false;
  String? _error;

  ServiciosViewModel(this._serviciosService);

  List<Servicio> get servicios => _servicios;
  bool get cargando => _cargando;
  String? get error => _error;

  Future<void> cargarServicios() async {
    _cargando = true;
    _error = null;
    notifyListeners();

    try {
      _servicios = await _serviciosService.obtenerServicios();
    } catch (e) {
      _error = e.toString();
    } finally {
      _cargando = false;
      notifyListeners();
    }
```

En esta clase es donde se da autenticación de los usuarios

auth_service.dart

lib > Service > auth_service.dart > AuthService > _agregarUsuariosPrueba

```dart
import 'package:flutter/material.dart';
import 'package:flutter_application_appdate2/MVVM/Models/usuario.dart';

class AuthService with ChangeNotifier {
  final List<Usuario> _usuarios = [];
  Usuario? _usuarioActual;
  bool _cargando = false;

  bool get cargando => _cargando;
  List<Usuario> get usuarios => _usuarios;
  Usuario? get usuarioActual => _usuarioActual;

  AuthService() {
    _agregarUsuariosPrueba();
    debugPrint('AuthService initialized with ${_usuarios.length} users');
  }

  void _agregarUsuariosPrueba() {
    _usuarios.clear();
    _usuarios.addAll([
      // Usuario Administrador
      Usuario(
        id: 'admin01',
        nombre: 'Boga',
        correo: 'boga@admin.com',
        telefono: '6861112233',
        tipoUsuario: 'admin',
        pasword: 'AdminBoga123',
      ),
```

servicios_service.dart

lib > Service > servicios_service.dart > ...

```dart
class ServiciosService {
  }

  Future<List<Servicio>> obtenerServicios() async {
    await Future.delayed(const Duration(milliseconds: 500));
    return servicios.where((s) => s.activo).toList();
  }

  Future<List<Servicio>> obtenerServiciosPorProveedor(String proveedorId) async {
    await Future.delayed(const Duration(milliseconds: 500));
    return servicios.where((s) => s.proveedorId == proveedorId && s.activo).toList();
  }

  Future<Servicio> agregarServicio(Servicio servicio) async {
    await Future.delayed(const Duration(milliseconds: 500));
    final nuevoServicio = Servicio(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      proveedorId: servicio.proveedorId,
      nombre: servicio.nombre,
      descripcion: servicio.descripcion,
      duracion: servicio.duracion,
      precio: servicio.precio,
      categoria: servicio.categoria,
      nombreProveedor: servicio.nombreProveedor,
    ); // Servicio
    servicios.add(nuevoServicio);
    return nuevoServicio;
  }

  // --- FUNCIÓN PARA ELIMINAR UN SERVICIO ---
  Future<void> eliminarServicio(String servicioId) async {
    await Future.delayed(const Duration(milliseconds: 500));
```

Aqui se maneja los servicios que se dan de altas.

Aquí se validan los caracteres que se usan en el regristro de usuario

validadores.dart

lib > Service > validadores.dart > ...

```dart

class Validadores {
 //correo Electrónico ---
  static String? validarCorreo(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'El correo electrónico es requerido.';
    }

    final regex = RegExp(r'^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$');
    if (!regex.hasMatch(value.trim())) {
      return 'Por favor, introduce un correo electrónico válido.';
    }
    return null;
  }

  // contrasena
  static String? validarContrasena(String? value) {
    if (value == null || value.isEmpty) {
      return 'La contraseña es requerida.';
    }
    if (value.length < 8) {
      return 'Debe tener al menos 8 caracteres.';
    }
    if (!value.contains(RegExp(r'[A-Z]'))) {
      return 'Debe tener al menos una letra mayúscula (A-Z).';
    }
    if (!value.contains(RegExp(r'[a-z]'))) {
      return 'Debe tener al menos una letra minúscula (a-z).';
    }
    if (!value.contains(RegExp(r'[0-9]'))) {
      return 'Debe tener al menos un número (0-9).';
```

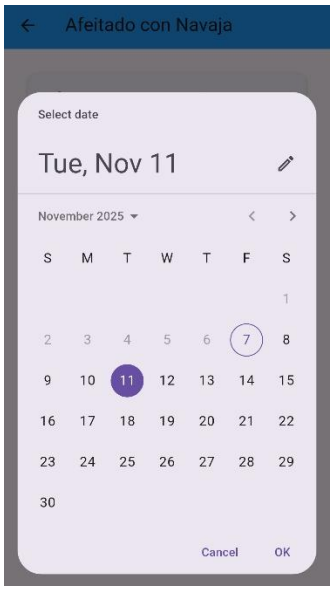
Tercer Sprint.

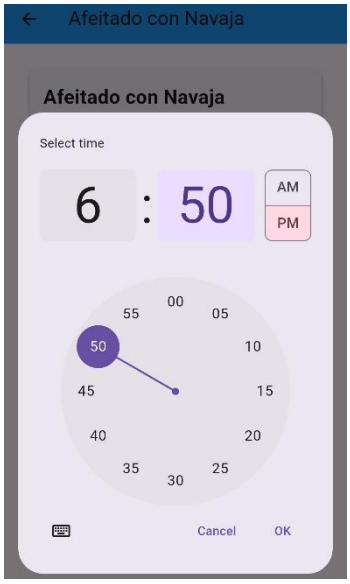
En la presentación del tercer sprint se hizo la entrega de los avances de la aplicación, la fecha establecida en el calendario es del 07 de noviembre del 2025, para esta entrega del tercer sprint, se estableció que se entregarían los siguientes datos:

- **Sprint 3: Núcleo de Citas**

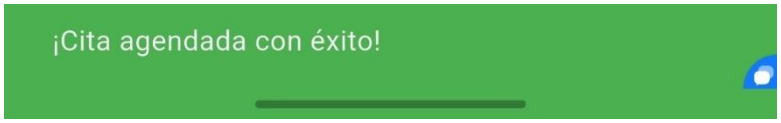
✓ Agendado de citas: La aplicación permitirá que los usuarios puedan programar citas con los servicios disponibles por los proveedores, los cuales incluirá este apartado, fecha, hora y tipo de servicio seleccionado.

Al darle clic, en el apartado de agendar cita, se mandará a la pantalla de calendario, la cual te hará seleccionar el día y posteriormente la hora, al seleccionar el horario, saldrá una leyenda de color verde, donde se confirma su cita.

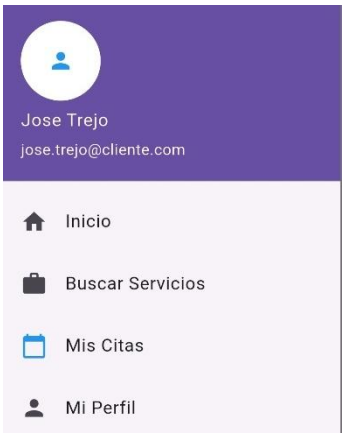




- Confirmación de cita:



- ✓ Panel de citas vigentes: Los usuarios contaran con un panel de control, el cual permitirá visualizar todas las citas que el usuario cuenta, en este apartado se mirara la cita y se visualizara la fecha, hora, proveedor, estado de cita.



En la pantalla sala el apartado de citas, en el cual al darle clic mandara la pantalla de mis citas.

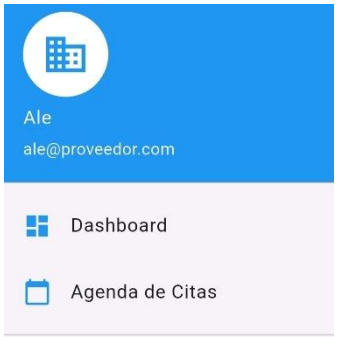
- ✓ Cancelación de citas: Al usuario se le ofrecerá la opción de cancelar su cita, esta opción se puede realizar por medio del cliente o del proveedor.

Cliente

Al mostrar el panel de citas, se puede observar el icono de la x, al darle clic se podrán cancelar la cita, mandando una pantalla de confirmación, aceptar la cancelación, el estado de la cita cambiara a cancelado.

- **Cita cancelada:**

Proveedor:



En el caso del proveedor, es el mismo, se podrá observar las citas por medio del panel del agendado, después al darle clic al icono de la x, se mostrará la pantalla de confirmación y al aceptar quedará la cita cancelada, después el estado de la cita cambiará a cancelado.

Código del 3er Sprint

En la tercera semana de entrega se desarrolló el código del tercer sprint, el objetivo con el que se trabajo fue poder establecer el núcleo de citas, en el cual se desarrollo el que el usuario pueda cancelar, agendar, visualizar sus citas.

```
main.dart    cita.dart ×
lib > MVVM > Models > cita.dart > EstadoCita
1   enum EstadoCita { Confirmada, Cancelada, Completada, P
2
3   class Cita {
4     final String id;
5     final String clienteId;
6     final String clienteNombre;
7     final String proveedorId;
8     final String servicioId;
9     final String servicioNombre;
10    final DateTime fecha;
11    final String hora;
12    final int duracion;
13    final double precio;
14    EstadoCita estado;
15
16    Cita({
17      required this.id,
18      required this.clienteId,
19      required this.clienteNombre,
20      required this.proveedorId,
21      required this.servicioId,
22      required this.servicioNombre,
23      required this.fecha,
24      required this.hora,
25      required this.duracion,
26      required this.precio,
27      this.estado = EstadoCita.Confirmada,
28    });
29  }
```

Se agrego la clase cita, en la cual se define la información en la cual se puede observar es la del cliente, proveedor, servicio, fecha, duración, precio y estado, al insertar la parte de confirmada, permitimos que se puede elaborar la tarea de organizar y controlar citas.

Después se llamó esta la clase pagina_citas en la clase dashboard_Cliente.

```
lib > MVVM > View > dashboard_cliente.dart > DashboardCliente > build
1   import 'package:flutter/material.dart';
2   import 'package:provider/provider.dart';
3   import 'package:flutter_application_appdate2/Service/auth_service.dart';
4   import 'package:flutter_application_appdate2/MVVM/Models/usuario.dart';
5   import 'package:flutter_application_appdate2/MVVM/View/pagina_citas.dart';
6   import 'package:flutter_application_appdate2/MVVM/View/pagina_servicios.dart';
7   import 'package:flutter_application_appdate2/MVVM/View/pagina_perfil.dart';
8
9   class DashboardCliente extends StatelessWidget {
10    final Usuario usuario;
11
12    const DashboardCliente({super.key, required this.usuario});
13
14    @override
15    Widget build(BuildContext context) {
16      final authService = Provider.of<AuthService>(context, listen: false);
```

```
main.dart    dashboard_cliente.dart ×
lib > MVVM > View > dashboard_cliente.dart > DashboardCliente > build
9   class DashboardCliente extends StatelessWidget {
15    Widget build(BuildContext context) {
46              title: const Text('Buscar Servicios'),
47              onTap: () {
48                Navigator.pop(context);
49                Navigator.push(context, MaterialPageRoute(builder: (context) => const PaginaServicios()));
50              },
51            ), // ListTile
52            ListTile(
53              leading: const Icon(Icons.calendar_today, color: Colors.blue),
54              title: const Text('Mis Citas'),
55              onTap: () {
56                Navigator.pop(context);
57                Navigator.push(context, MaterialPageRoute(builder: (context) => const PaginaCitas()));
58              },
59            ), // ListTile
60            ListTile(
61              leading: const Icon(Icons.person),
62              title: const Text('Mi Perfil'),
63              onTap: () {
64                Navigator.pop(context);
65                Navigator.push(context, MaterialPageRoute(builder: (context) => const PaginaPerfil()));
66              },
67            ), // ListTile
68            const Divider(),
69            ListTile(
70              leading: const Icon(Icons.logout),
71              title: const Text('Cerrar Sesión'),
72              onTap: () => authService.cerrarSesion(),
```

De ahí se hizo una modificación en la clase dashboard_Cliente, en la cual, se realiza el menú lateral que nos muestre las citas del usuario.

Después se llamó esta la clase pagina_citas en la clase dashboard_Proveedor.

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:flutter_application_appdate2/Service/auth_service.dart';
import 'package:flutter_application_appdate2/MVVM/Models/usuario.dart';
import 'package:flutter_application_appdate2/MVVM/View/pagina_citas.dart';
import 'package:flutter_application_appdate2/MVVM/View/pagina_servicios.dart';
import 'package:flutter_application_appdate2/MVVM/View/agregar_servicio.dart';
import 'package:flutter_application_appdate2/MVVM/View/pagina_perfil.dart';
import 'package:flutter_application_appdate2/MVVM/View/agregar_horario.dart';
//import 'package:flutter_application_appdate2/MVVM/View/pagina_horario.dart';

class DashboardProveedor extends StatelessWidget {
  final Usuario usuario;

  const DashboardProveedor({super.key, required this.usuario});
```

```dart
class DashboardProveedor extends StatelessWidget {
  Widget _buildBody(BuildContext context) {
            style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
            textAlign: TextAlign.center,
          ), // Text
          const SizedBox(height: 10),
          const Text(
            'Panel de Control - Proveedor',
            style: TextStyle(fontSize: 16, color: Colors.grey),
            textAlign: TextAlign.center,
          ), // Text
          const SizedBox(height: 40),
           _buildActionButton(
            context: context,
            icon: Icons.calendar_today,
            label: 'Ver Agenda de Citas',
            onPressed: () {
              Navigator.push(context, MaterialPageRoute(builder: (_) => const PaginaCitas()));
            },
          ),
          const SizedBox(height: 12),
          _buildActionButton(
            context: context,
            icon: Icons.add_business,
            label: 'Agregar Nuevo Servicio',
            onPressed: () {
              Navigator.push(context, MaterialPageRoute(builder: (_) => const PaginaAgregarServicio()));
            },
```

En esta parte del código podemos ver un botón personalizado un icono de calendario por ver agenda de citas.

Se creo la página de citas donde el objetivo se muestra las citas del usuario, donde se mostrará el indicador donde se pueda observar la lista de citas.

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:flutter_application_appdate2/MVVM/Models/servicio.dart';
//import 'package:flutter_application_appdate2/MVVM/Models/horario.dart';
import 'package:flutter_application_appdate2/Service/auth_service.dart';
import 'package:flutter_application_appdate2/Service/horario_service.dart';
import 'package:flutter_application_appdate2/MVVM/View_Models/citas_view_model.dart';
import 'package:flutter_application_appdate2/MVVM/View_Models/horario_view_model.dart';

class PaginaDetalleServicio extends StatelessWidget {
  final Servicio servicio;

  const PaginaDetalleServicio({super.key, required this.servicio});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => HorarioViewModel(HorarioService())
        ..cargarHorariosProveedor(servicio.proveedorId),
      child: Scaffold(
        appBar: AppBar(
          title: Text(servicio.nombre),
          backgroundColor: Colors.blue,
        ), // AppBar
        body: SingleChildScrollView(
          padding: const EdgeInsets.all(24.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Card(
                child: Padding(
                  padding: const EdgeInsets.all(16.0),
```

```dart
import 'package:flutter/material.dart';
import 'package:flutter_application_appdate2/MVVM/Models/cita.dart';
import 'package:flutter_application_appdate2/MVVM/Models/servicio.dart';
import 'package:flutter_application_appdate2/MVVM/Models/usuario.dart';
import 'package:flutter_application_appdate2/Service/citas_service.dart';

class CitasViewModel with ChangeNotifier {
  final CitasService _citasService;

  List<Cita> _citas = [];
  bool _cargando = false;
  String? _error;

  CitasViewModel(this._citasService);

  List<Cita> get citas => _citas;
  bool get cargando => _cargando;
  String? get error => _error;

  Future<void> cargarCitas(Usuario usuario) async {
    _cargando = true;
    notifyListeners();
    try {
      if (usuario.esCliente) {
        _citas = await _citasService.obtenerCitasPorCliente(usuario.id);
      } else if (usuario.esProveedor) {
        _citas = await _citasService.obtenerCitasPorProveedor(usuario.id);
      }
    } catch (e) {
      _error = e.toString();
    } finally {
      _cargando = false;
```

En esta clase se gestiona el estado y la lógica de las citas, en él se puede agendar, cancelar utilizando los servicios de la página citaservice.

En esta clase se muestra visualmente como se mirará la tarjeta de cita ya con el nombre de servicio, fecha, estado y los datos de los clientes.

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:flutter_application_appdate2/MVVM/Models/cita.dart';
import 'package:flutter_application_appdate2/MVVM/View_Models/citas_view_model.dart';
import 'package:flutter_application_appdate2/Service/auth_service.dart';

class TarjetaCita extends StatelessWidget {
  final Cita cita;
  const TarjetaCita({super.key, required this.cita});

  @override
  Widget build(BuildContext context) {
    final esCliente = context.read<AuthService>().usuarioActual?.esCliente ?? true;

    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      color: cita.estado == EstadoCita.Cancelada ? Colors.grey[300] : Colors.white,
      child: ListTile(
        leading: Icon(
          Icons.event_available,
          color: cita.estado == EstadoCita.Cancelada ? Colors.grey : Colors.blue,
        ), // Icon
        title: Text(cita.servicioNombre, style: const TextStyle(fontWeight: FontWeight.bold)),
        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(esCliente ? 'Con: ${cita.servicioNombre}' : 'Cliente: ${cita.clienteNombre}'),
            Text('Fecha: ${cita.fecha.day}/${cita.fecha.month}/${cita.fecha.year} a las ${cita.hora}'),
            Text('Estado: ${cita.estado.name}', style: TextStyle(fontWeight: FontWeight.bold, color: _getColorEst
          ],
```

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:flutter_application_appdate2/MVVM/View_Models/citas_view_model.dart';
import 'package:flutter_application_appdate2/MVVM/View_Models/tarjeta_cita.dart';
import 'package:flutter_application_appdate2/Service/auth_service.dart';

class PaginaCitas extends StatefulWidget {
  const PaginaCitas({super.key});

  @override
  State<PaginaCitas> createState() => _PaginaCitasState();
}

class _PaginaCitasState extends State<PaginaCitas> {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      final usuario = context.read<AuthService>().usuarioActual;
      if (usuario != null) {
        context.read<CitasViewModel>().cargarCitas(usuario);
      }
    });
  }

  @override
  Widget build(BuildContext context) {
    final viewModel = context.watch<CitasViewModel>();

    return Scaffold(
      appBar: AppBar(
        title: const Text('Mis Citas'),
```

En esta pagina se definirá la parte donde se cargara el inicio de la página, donde se podrá visualizar al usuario y se cargaran las citas que se agendaron.

Esta clase, ayuda para simular el servicio de gestionar citas, en la cual se puede dar inicio, agregar una cita de prueba, además de obtener la citas por cliente o proveedor.

main.dart × citas_service.dart ×

lib > Service > citas_service.dart > CitasService > _agregarCitasDePrueba

```dart

import 'package:flutter_application_appdate2/MVVM/Models/cita.dart';

class CitasService {
  final List<Cita> _citas = [];

  CitasService() {
    _agregarCitasDePrueba();
  }

  void _agregarCitasDePrueba() {
    _citas.add(
      Cita(
        id: 'cita001',
        clienteId: 'cli01',
        clienteNombre: 'Jose Trejo',
        proveedorId: 'prov01',
        servicioId: '1',
        servicioNombre: 'Corte de Cabello Clásico',
        fecha: DateTime.now().add(const Duration(days: 2)),
        hora: '15:30',
        duracion: 30,
        precio: 150.0,
        estado: EstadoCita.Confirmada,
      ), // Cita
    );
  }
```